

Context Pruning Script

Context Pruning is the process of trimming product data before sending it to an AI model, particularly one using a Large Language Model (LLM) like OpenAI GPT. This pruning process typically occurs when the system prepares `product_data` to be inserted into the System Prompt.

There are five technical reasons why I need to do data pruning:

- 1. Saving Costs (Token Efficiency):** APIs like OpenAI's charge based on the number of tokens sent and received. Sending product data with fields that aren't useful to AI, such as long local image file paths or internal IDs, will waste tokens and money, as they're handled differently than AI Studio.
- 2. Expensive API Fees:** OpenAI charges based on the number of tokens sent and received. The longer the data, the higher the operational costs.
- 3. Information Overload:** If AI is given too much irrelevant information, it will become confused, lose focus, and ultimately give inaccurate answers (hallucinations).
- 4. Prevent AI Hallucinations and Confusion:** AI can become confused if it receives too much irrelevant data. By providing only the name, price, and description, the AI can focus on answering the question much more accurately.
- 5. Hiding Confidential/Irrelevant Data:** If a product is out of stock or hidden, the AI won't reveal it to anyone else. Pruning ensures this confidential data is removed from the prompt.

How Context Pruning Works:

Here are the three layers of pruning that occur:

1. Product Status Filtering

The first step is to remove products that should not be seen by the AI or the user.

- **Code Logic:** The script will check for the `hidden: true` property on each product in `products.json`.
- **Impact:** If a product is out of stock, the script will remove it from the list sent to the AI. The AI will never know the product is still available, so it will never offer it to the user.

1. Data Attribute Trimming

Even though the product is live and will be sent to AI, not all of the information in the product's JSON is useful to the language model.

- **Code Logic:** The script will create a new placeholder for each product. After passing the entire JSON object, the script will retrieve only the relevant keys to answer the query, such as name, price, rating, and description.
- **Impact:** An internal system attribute will discard any HTML code elements that might be embedded within the image link if it's not using image recognition. This will save tokens.

2. Chat History Trimming (Sliding Window)

Conversational memory is crucial for AI to remember context, but conversations that are too long will cause it to waste tokens.

- **Code Logic:** The script applies a sliding window technique to chat_history. It will only retrieve the last few conversations, for example, the last five questions and answers, and discard chat history that is too old.
- **Impact:** The AI will still feel smarter because it remembers the context of the current conversation, but the system doesn't need to remember the burden of the conversation from 2 hours ago.

Below is a comparison table of the data loads sent to AI:

Metric	Before Context Pruning	After Context Pruning
Products Shipped	All products in products.json.	Only active and relevant products have ready status.
Attributes per Product	Contains links, metadata, and system code that are useless to AI.	Purely just Name, Price, Description, and Rating.
Chat History	Brings 50+ messages from the moment the user opens the web.	Only brings the last 3-5 interactions as immediate context.
Efficiency and Cost	condition touches the token limit, the cost is very expensive.	Very lightweight, much faster AI response, and API cost-effective.

The use of this script is driven by several technical and economic challenges in the artificial intelligence ecosystem:

- **Context Window Limits:** Each LLM has a maximum limit on the number of tokens (pieces of words) it can process at a time (e.g., 8,000, 32,000, or 128,000 tokens). If the conversation history or reference document is too long, the model will either reject the request (resulting in an error) or randomly truncate the text. Pruning scripts systematically prevent this.
- **Cost Reduction:** Most commercial LLM APIs charge based on the number of tokens sent (input) and generated (output). Re-sending entire lengthy conversation histories—including greetings or outdated chats—would be a significant drain on your budget.
- **Speed and Latency (Latency Optimization):** The more tokens a model has to analyze, the longer it will take to compute. Trimming unnecessary context will directly speed up the AI's response time to users.
- **Avoiding the Lost in the Middle Phenomenon:** Research shows that LLMs have a weakness where they tend to forget or ignore information buried in the middle of very long inputs (they are biased towards information at the beginning and end of the prompt). By keeping the context concise and concise, the AI's hallucination rate decreases and its accuracy increases.